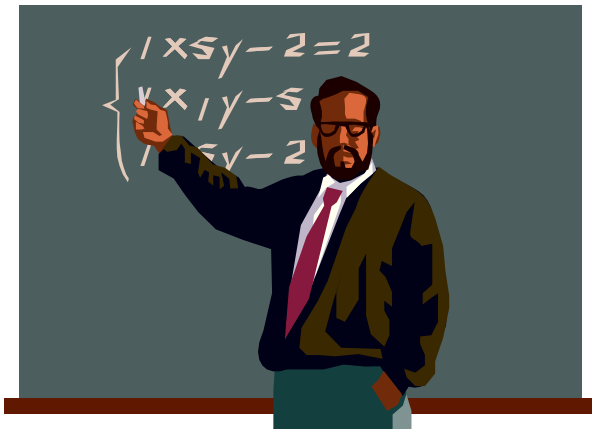
1장 자료구조와 알고리즘

Q & A

- 언제라도 질문하세요
  모르면 외우면 되지만 코딩 안되면 꼭 질문
  ➔ 중간, 기말 오픈북 코딩시험, 인터넷차단
  1. 한성e-class 질의응답게시판
- 강사의 1번 선생님은 여러분들의 질문



© 생능출판사 2019

앞으로 나는 할 수 있다

1. 나는 자료구조와 알고리즘 사이의 상호 작용을 설명할 수 있다.
2. 나는 알고리즘의 성능분석을 할 수 있어 내 알고리즘이 타인의 알고리즘보다 빠르다고 주장할 수 있다.
   (내 코드가 최고입니다~)

© 생능출판사 2019

자료구조와 알고리즘

- 프로그램 = 자료구조(변수) + 알고리즘(실행문)

자료구조

scores[]

80	70	90	···	30

알고리즘

```
largest←scores[0]
for i←1 to N-1 do
  if scores[i]>largest
     then largest←scores[i]
return largest
```

© 생능출판사 2019

```c
#define MAX_ELEMENTS 100
int scores[MAX_ELEMENTS]; // 자료구조
int get_max_score(int n)
{
        int i, largest;
        largest = scores[0];  // 알고리즘
        for (i = 1; i<n; i++) {
                if (scores[i] > largest) {
                        largest = scores[i];
                }
        }
        return largest;
}
```


© 생능출판사 2019

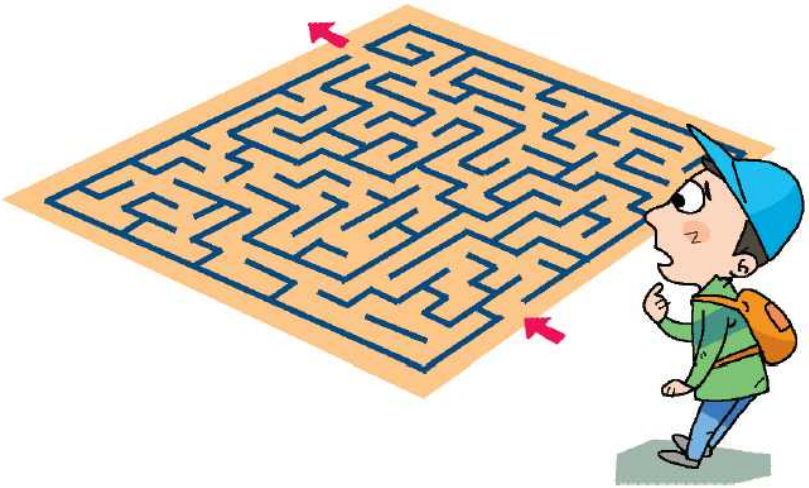
알고리즘

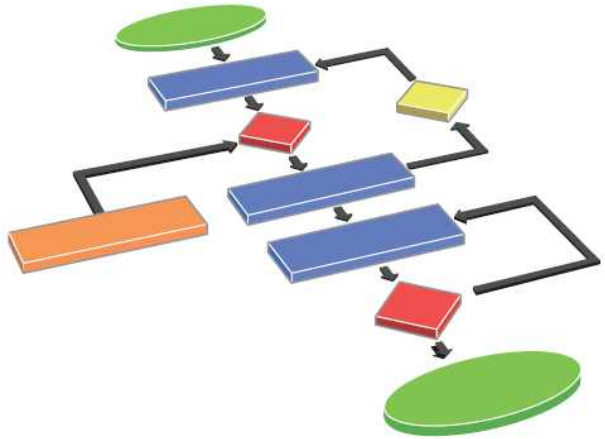
- 알고리즘(algorithm): 컴퓨터로 문제를 풀기 위한 단계적인 절차

© 생능출판사 2019

알고리즘의 기술 방법

- 영어나 한국어와 같은 자연어
- 흐름도(flow chart)
- 의사 코드(pseudo-code)
- 프로그래밍 언어





© 생능출판사 2019

자연어로 표기된 알고리즘

- 인간이 읽기가 쉽다.
- 그러나 자연어의 단어들을 정확하게 정의하지 않으면 의미 전달이 모호해질 우려가 있다.

(예) 배열에서 최대값 찾기 알고리즘

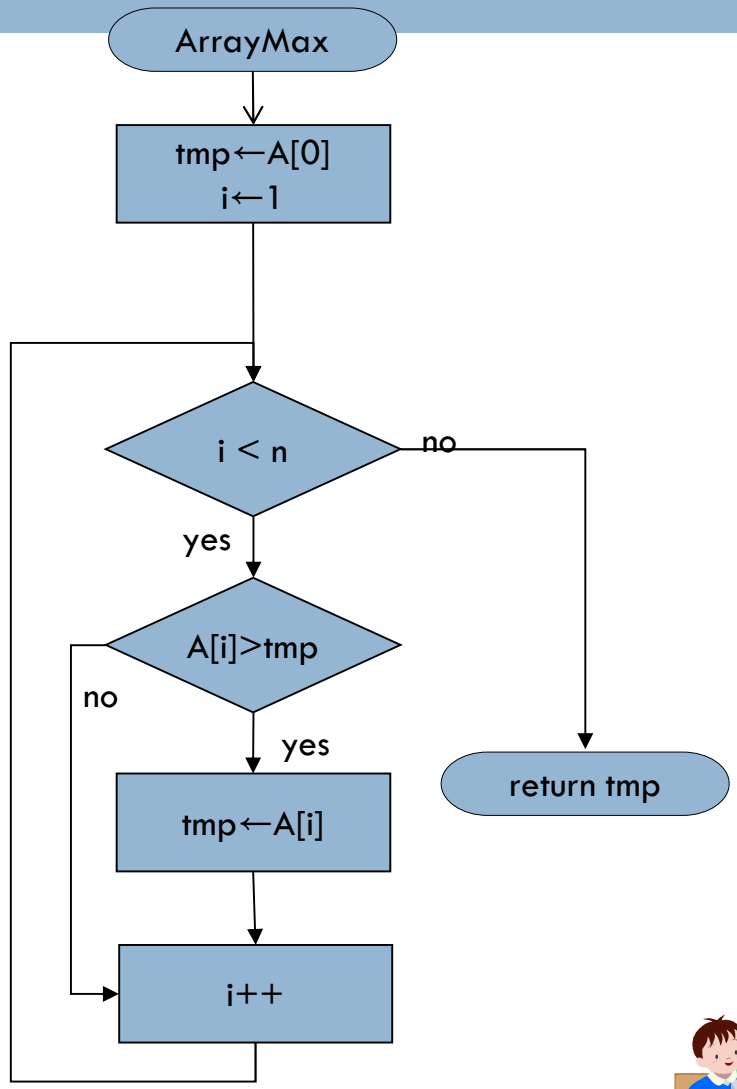
ArrayMax(list, n)

1. 배열 list의 첫번째 요소를 변수 tmp에 복사
2. 배열 list의 다음 요소들을 차례대로 tmp와 비교하면 더 크면 tmp로 복사
3. 배열 list의 모든 요소를 비교했으면 tmp를 반환

© 생능출판사 2019

흐름도로 표기된 알고리즘

- 직관적이고 이해하기 쉬운 알고리즘 기술 방법
- 그러나 복잡한 알고리즘의 경우, 상당히 복잡해짐.



© 생능출판사 2019

유사코드로 표현된 알고리즘

- 알고리즘 기술에 가장 많이 사용
- 프로그램을 구현할 때의 여러 가지 문제들을 감출 수 있다.
  즉 알고리즘의 핵심적인 내용에만 집중할 수 있다.

```
ArrayMax(list, N):
    largest←list[0]
    for i←1 to N-1 do
        if list[i]>largest
            then largest←list[i]
    return largest
```

© 생능출판사 2019

C언어로 표현된 알고리즘

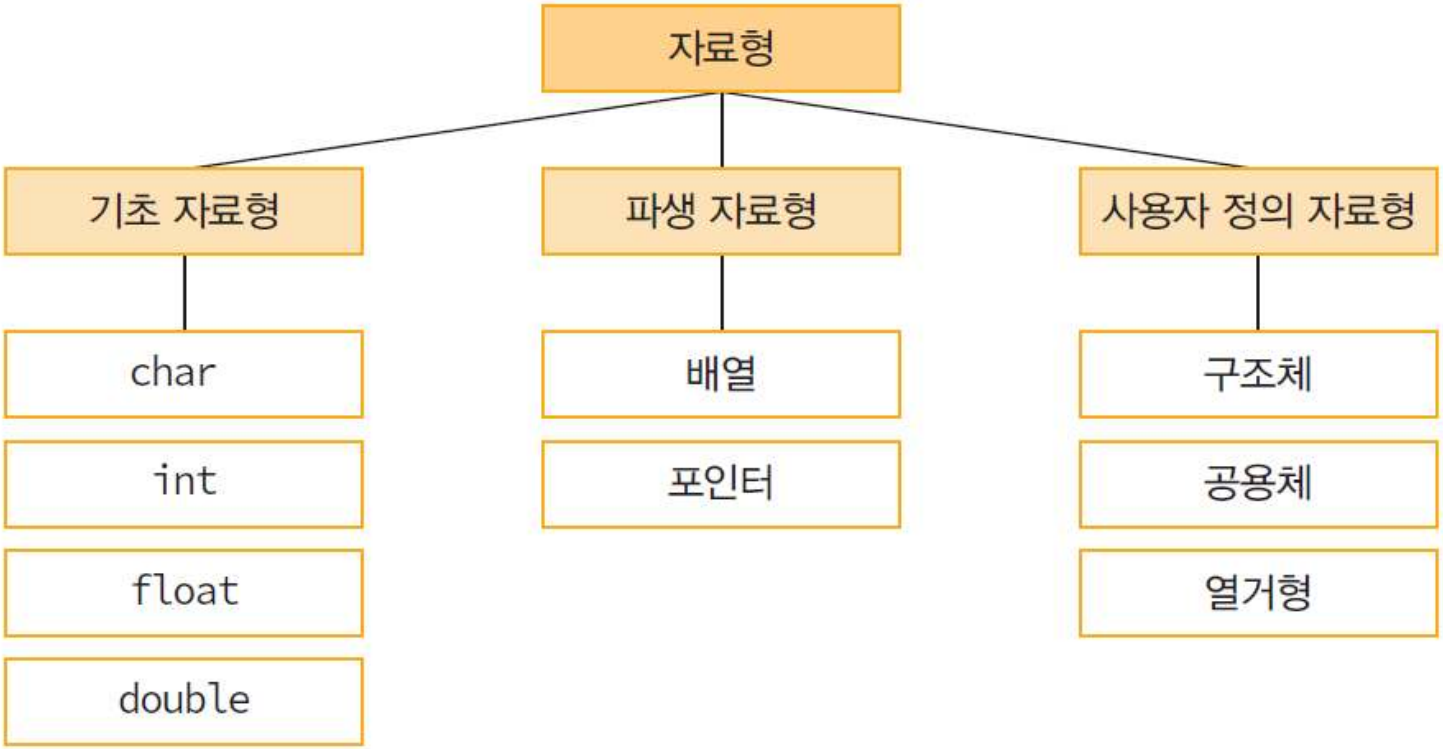
- 알고리즘의 가장 정확한 기술이 가능
- 반면 실제 구현 시, 많은 구체적인 사항들이 알고리즘의 핵심적인 내용에 대한 이해를 방해할 수 있다.

```
#define MAX_ELEMENTS 100
int score[MAX_ELEMENTS];
int find_max_score(int n)
{
        int i, tmp;
        tmp=score[0];
        for(i=1;i<n;i++){
                if( score[i] > tmp ){
                        tmp = score[i];
                }
        }
        return tmp;
}
```

© 생능출판사 2019

자료형 (자료구조)

- 자료형(data type): “데이터의 종류”
- 정수, 실수, 문자열 등이 기초적인 자료형의 예



© 생능출판사 2019

자료형 (자료구조)

- 자료형(data type)
  - 데이터의 집합과 관련된 연산의 집합으로 생각

int 자료형

데이터: {-INT_MIN, ..., -2, -1. 0, 1, 2, ..., INT_MAX }

연산: +, -, *, /, %, ==, >, <

© 생능출판사 2019

추상 데이터 타입

- 추상 데이터 타입(ADT: Abstract Data Type)
  - C언어 문법에서는 없고 OO언어의 Abstract class와 유사개념 stack과 같은 새로운 데이터타입(자료형, 자료구조)에는 stack 에 필요한 연산도 같이 정의가 필요하기에 ADT라는 개념을 교재에서 도입함
  - 데이터 타입을 추상적(구체적이 아닌 요약적)으로 정의한 것
  - 데이터나 연산이 **무엇(what)**인가는 정의되지만 데이터나 연산을 **어떻게(how)** 컴퓨터 상에서 구현할 것인지는 정의되지 않는다.

© 생능출판사 2019

C++의 class와 비교

- 객체: C언어로 자료구조 구현방법 vs. C++에서는 new 동적생성
- 연산: ADT의 필요한 연산(함수) vs. C++에서는 멤버함수
- C언어는 OO가 아니지만 stack과 같은 새로운 자료구조 생성시 그 자료구조에 필요한 연산도 같이 정의가 필요

< stack의 ADT>
·객체: 0개 이상의 원소를 가지는 유한 선형 리스트
·연산:
▪ push(s, item) ::=
if(is_full(s)) return ERROR_STACKFULL;
else 스택의 맨 위에 item을 추가한다.

클래스 정의

```
class 클래스이름 {
    자료형 멤버변수1;
    자료형 멤버변수2;

    반환형 멤버함수1();
    반환형 멤버함수2();
};
```

멤버 변수

멤버 함수 선언부

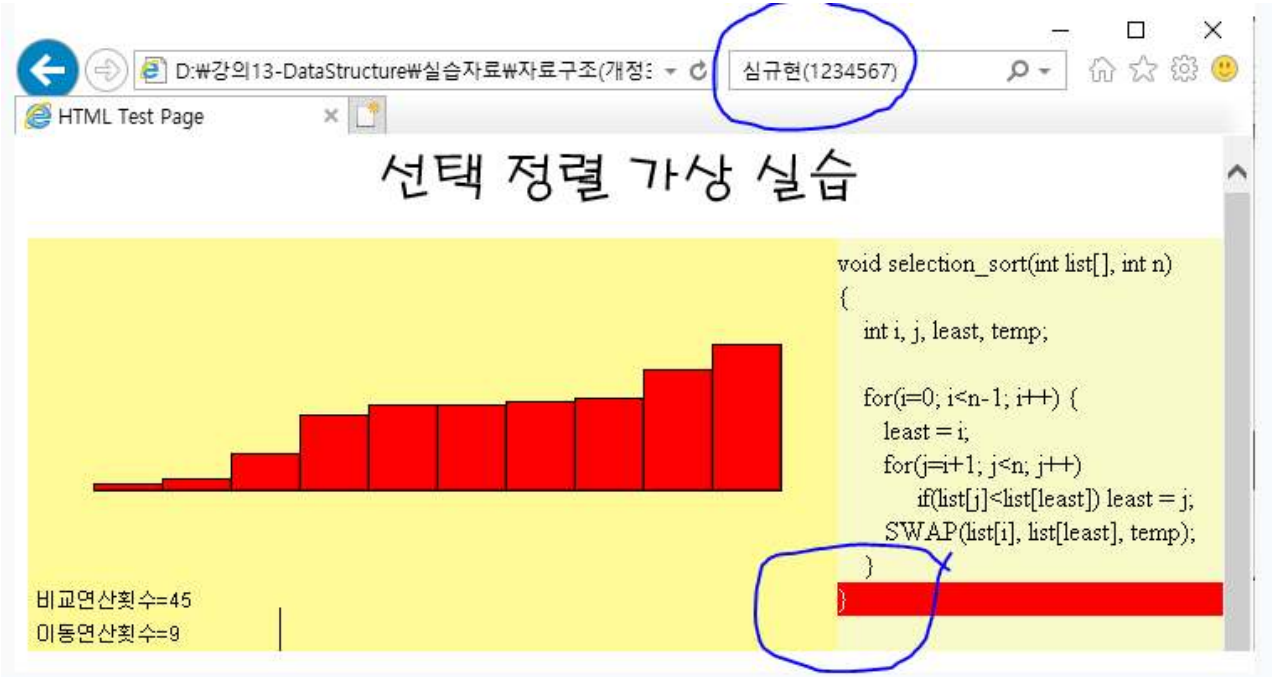
자료구조와 알고리즘의 상호 작용 (가상실습)

- 가상실습을 통해
  알고리즘이 진행하며 자료구조를 변화시키고
  변화된 자료구조가 알고리즘의 진행방향을 변화시키는 것을 코드 줄 단위로 확인 가능
- javaApplet으로 구현되어 IE에서 작년에는 실행 되었으나 금년 win10 20H2버전에는 실행이 안됨. 시연만 하겠음
- Visual studio에서 자료구조와 알고리즘을 수강생들과 같이 보겠음

© 생능출판사 2019

자료구조와 알고리즘의 상호 작용 (가상실습)

- 가상실습을 통해
  알고리즘이 진행하며 자료구조를 변화시키고
  변화된 자료구조가 알고리즘의 진행방향을 변화시키는
  것을 코드 줄 단위로 확인 가능



© 생능출판사 2019

알고리즘의 성능분석 (내 알고리즘이 제일 빠른가 ?)

- 알고리즘의 성능 분석 기법
  - 수행 시간 측정
    - 두개의 알고리즘의 실제 수행 시간을 측정하는 것
    - 실제로 구현하는 것이 필요
    - 동일한 하드웨어를 사용하여야 함
  - 알고리즘의 복잡도 분석
    - 직접 구현하지 않고서도 수행 시간을 분석하는 것
    - 알고리즘이 수행하는 연산의 횟수를 측정하여 비교
    - 일반적으로 연산의 횟수는 n(입력자료의 개수)의 함수

© 생능출판사 2019

교수님~ 성능 분석을 왜 하나요?

- 여러분은 2학년1학기로서 1학년에서 C언어(컴퓨터프로그래밍), JS(웹프로그래밍기초), Python(교양필수) 배움
- 2학년에서는 C++과 Java를 배움
- 이제 어떻게든 코딩할 수 있음
- 자료구조과목과 알고리즘과목은 성능이 우수한 코딩이 목표임
- 소비자들도 성능이 우수한 SW를 원하고 있음

© 생능출판사 2019

왜 프로그램의 효율성이 중요한가?

입력 자료의 개수	프로그램 A: n^2	프로그램 B: 2^n
$n = 6$	36초	64초
$n = 100$	10000초	2^{100}초$=4\times10^{22}$년

© 생능출판사 2019

수행시간측정

- 알고리즘을 프로그래밍 언어로 작성하여 실제 컴퓨터상에서 실행시킨 다음, 그 수행시간을 측정

© 생능출판사 2019

수행시간 측정 2가지 방법 [clock() 과 time()]

방법 #1	방법 #2

#include <time.h>
 	
start = clock();
...
stop = clock();
double duration = (double)(stop - start) / CLOCKS_PER_SEC;
	#include <time.h>
	
	start = time(NULL);
	...
	stop = time(NULL);
	double duration = (double) difftime(stop, start);

© 생능출판사 2019

수행시간측정

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main(void)
{
        clock_t start, stop;
        double duration;
        start = clock();        // 측정 시작
        for (int i = 0; i < 1000000; i++)            // 의미 없는 반복 루프
                ;
        stop = clock();         // 측정 종료
        duration = (double)(stop - start) / CLOCKS_PER_SEC;
        printf("수행시간은 %f초입니다.\n", duration);
        return 0;
}
```


© 생능출판사 2019

복잡도 분석

- 시간 복잡도는 알고리즘을 이루고 있는 연산들이 몇 번이나 수행되는지를 숫자로 표시

```
largest←scores[0]
for i←1 to N-1 do
    if scores[i]>largest
        then largest←scores[i]
return largest
```

© 생능출판사 2019

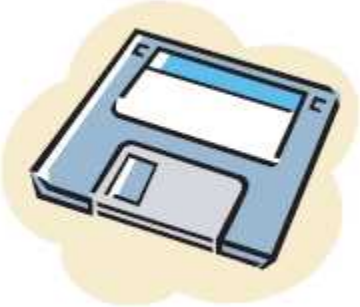
복잡도 분석의 종류

- 시간 복잡도(time complexity)
- 공간 복잡도(space complexity) : 사용되는 DRAM의 크기

© 생능출판사 2019

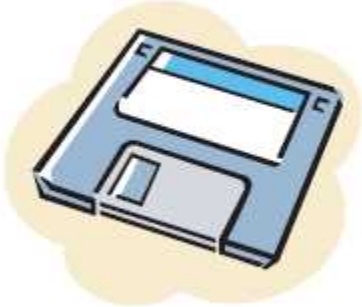
입력의 개수 고려

알고리즘 1



$3n+2$

알고리즘 2



$5n^2+6$

© 생능출판사 2019

복잡도 분석의 예

- 양의 정수 n을 n번 더하는 문제를 생각하여 보자.

알고리즘 A	알고리즘 B	알고리즘 C

sum ←n*n;	for i←1 to n do
	sum ←sum + n;
		for i←1 to n do
		for j←1 to n do
		sum ←sum + 1;

© 생능출판사 2019

알고리즘의 비교

알고리즘 A	알고리즘 B	알고리즘 C

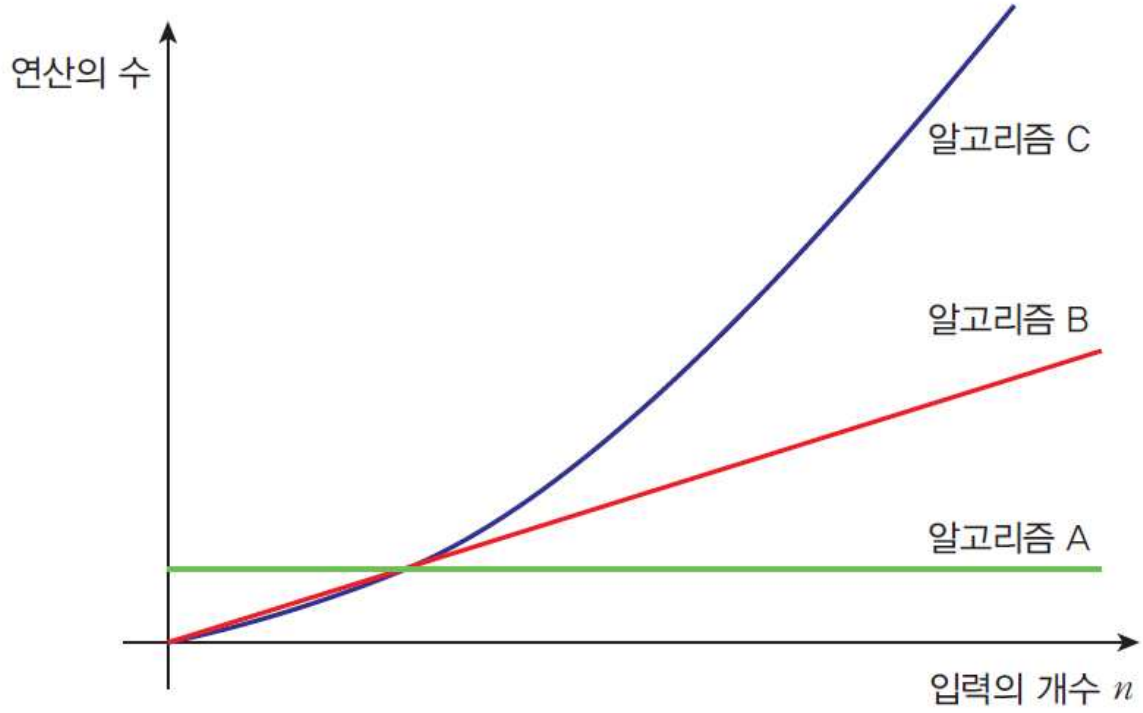
sum ←n*n;	for i←1 to n do
	sum ←sum + n;
		for i←1 to n do
		for j←1 to n do
		sum ←sum + 1;

	알고리즘 A	알고리즘 B	알고리즘 C
대입연산	1	n	$n * n$
덧셈연산		n	$n * n$
곱셈연산	1		
나눗셈연산			
전체연산수	2	$2n$	$2n^2$

© 생능출판사 2019

연산의 횟수를 그래프로 표현

- 알고리즘 A를 배우는 것이 자료구조와 알고리즘과목 목표



© 생능출판사 2019

빅오 표기법

- 자료의 개수가 많은 경우에는 차수가 가장 큰 항이 가장 영향을 크게 미치고 다른 항들은 상대적으로 무시될 수 있다.

n=1000인 경우

$T(n)= n^2 + n + 1$

99.9%

0.1%

© 생능출판사 2019

빅오 표기법

- 빅오표기법: 연산의 횟수를 대략적(점근적)으로 표기한 것

- 두개의 함수 f(n)과 g(n)이 주어졌을 때,
  모든 $n \geq n_0$에 대하여 $|f(n)| \leq c|g(n)|$을 만족하는 2개의 상수 c와 n_0가 존재하면 f(n)=O(g(n))이다.

- 빅오는 함수의 상한을 표시한다.
  - (예) $n \geq 5$ 이면 $2n+1 < 10n$ 이므로 $2n+1 = O(n)$

© 생능출판사 2019

빅오 표기법의 예

예제 1.1 빅오 표기법

- $f(n)=5$이면 $O(1)$이다. 왜냐하면 $n_0=1$, $c=10$일 때, $n>1$에 대하여 $5 \leq 10 \cdot 1$이 되기 때문이다.
- $f(n)=2n+1$이면 $O(n)$이다. 왜냐하면 $n_0=2$, $c=3$일 때, $n>2$에 대하여 $2n+1 \leq 3n$이 되기 때문이다.
- $f(n)=3n^2+100$이면 $O(n^2)$이다. 왜냐하면 $n_0=100$, $c=5$일 때, $n>100$에 대하여 $3n^2+100 \leq 5n^2$이 되기 때문이다.
- $f(n)=5 \cdot 2^n+10n^2+100$이면 $O(2^n)$이다. 왜냐하면 $n_0=1000$, $c=10$일 때, $n>1000$에 대하여 $5 \cdot 2^n+10n^2+100 \leq 10 \cdot 2^n$이 되기 때문이다.

© 생능출판사 2019

빅오 표기법의 종류

- $O(1)$: 상수형
- $O(\log n)$: 로그형
- $O(n)$: 선형
- $O(n \log n)$: 선형로그형
- $O(n^2)$: 2차형
- $O(n^3)$: 3차형
- $O(2^n)$: 지수형
- $O(n!)$: 팩토리얼형

$f(n)$	$O(f(n))$
10	$O(1)$
$5n^2+6$	$O(n^2)$
$2n^3+1$	$O(n^3)$
$2n^3+5n^2+6$	$O(n^3)$

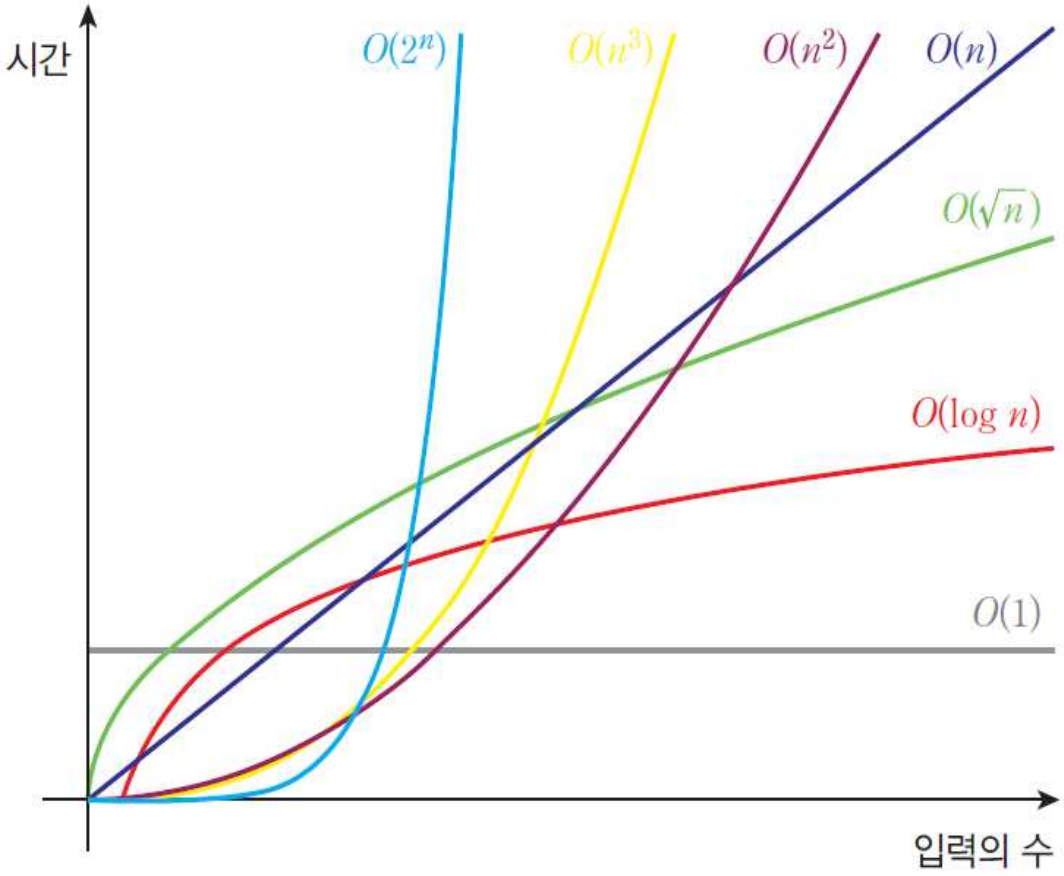
© 생능출판사 2019

빅오 표기법의 종류

시간복잡도	n					
	1	2	4	8	16	32
1	1	1	1	1	1	1
$\log n$	0	1	2	3	4	5
n	1	2	4	8	16	32
$n\log n$	0	2	8	24	64	160
n^2	1	4	16	64	256	1024
n^3	1	8	64	512	4096	32768
2^n	2	4	16	256	65536	4294967296
$n!$	1	2	24	40326	20922789888000	26313×10^{33}

© 생능출판사 2019

빅오표기법



- 빅데이터시대에는 코딩할 때 polynomial algorithm을 찾음 $O(n^k)$
- ➔ 고객들이 좋아함

© 생능출판사 2019

최선, 평균, 최악의 경우

- 알고리즘의 수행시간은 n(입력자료의 개수)뿐만 아니라 입력 자료 집합 내용에 따라 다를 수 있다.

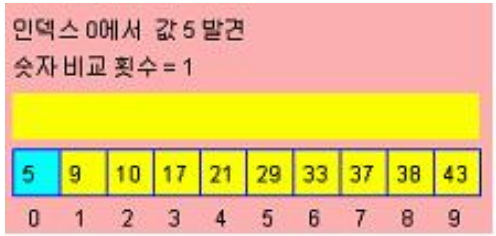
- **최선의 경우(best case):** 수행 시간이 가장 빠른 경우
- **평균의 경우(average case):** 수행시간이 평균적인 경우
- **최악의 경우(worst case):** 수행 시간이 가장 늦은 경우

© 생능출판사 2019

(예) 최선, 평균, 최악의 경우

- (예) 순차탐색
- 최선의 경우: 찾고자 하는 숫자가 맨 앞에 있는 경우

  ∴ O(1)



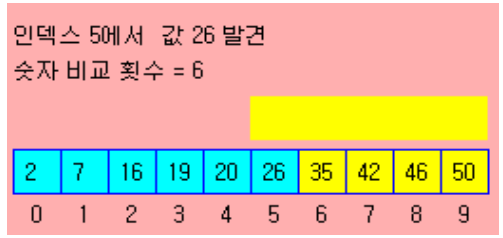
- 최악의 경우: 찾고자 하는 숫자가 맨 뒤에 있는 경우

  ∴ O(n)

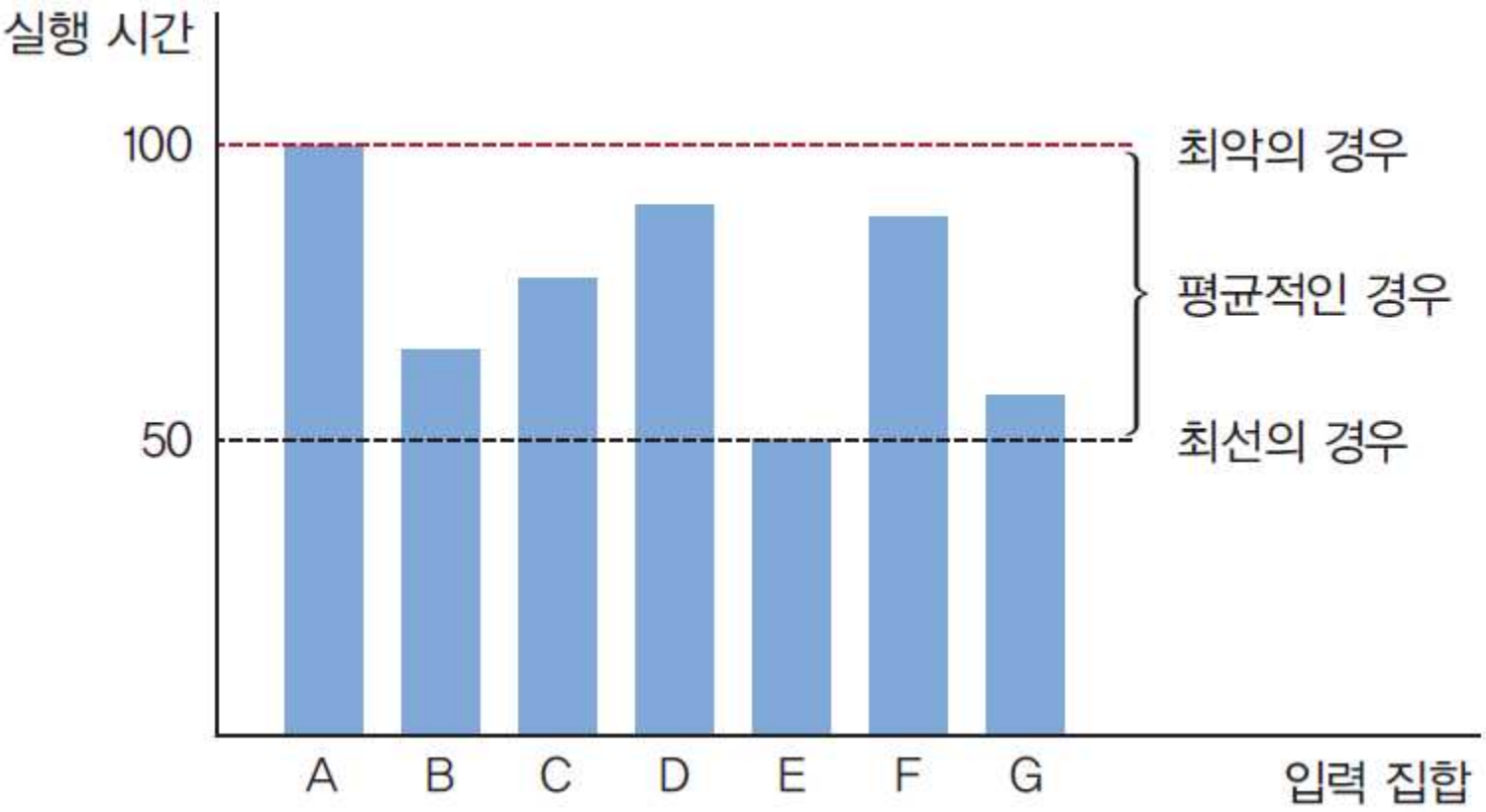
- 평균적인 경우: 각 요소들이 균일하게 탐색된다고 가정하면

  $(1+2+\cdots+n)/n=(n+1)/2$

  ∴ O(n)



© 생능출판사 2019



© 생능출판사 2019

최선, 평균, 최악의 경우

- 최선의 경우: 의미가 없는 경우가 많다.
- 평균적인 경우: 계산하기가 상당히 어려움.
- 최악의 경우: 가장 널리 사용된다. 계산하기 쉽고 응용에 따라서 중요한 의미를 가질 수도 있다.
  - (예) 비행기 관제업무, 게임, 로보틱스

© 생능출판사 2019

이제 나는 할 수 있다

1. 나는 자료구조와 알고리즘 사이의 상호 작용을 설명할 수 있다.
2. 나는 알고리즘의 성능분석을 할 수 있어 내 알고리즘이 타인의 알고리즘보다 빠르다고 주장할 수 있다.

* 1장 홀수번 실습문제 꼭 풀어보세요. (정답과 확인)

© 생능출판사 2019